
REST APIs

01

Recap



02

API Formats



03

REST w/ Django



04

REST w/ Spring boot



Agenda

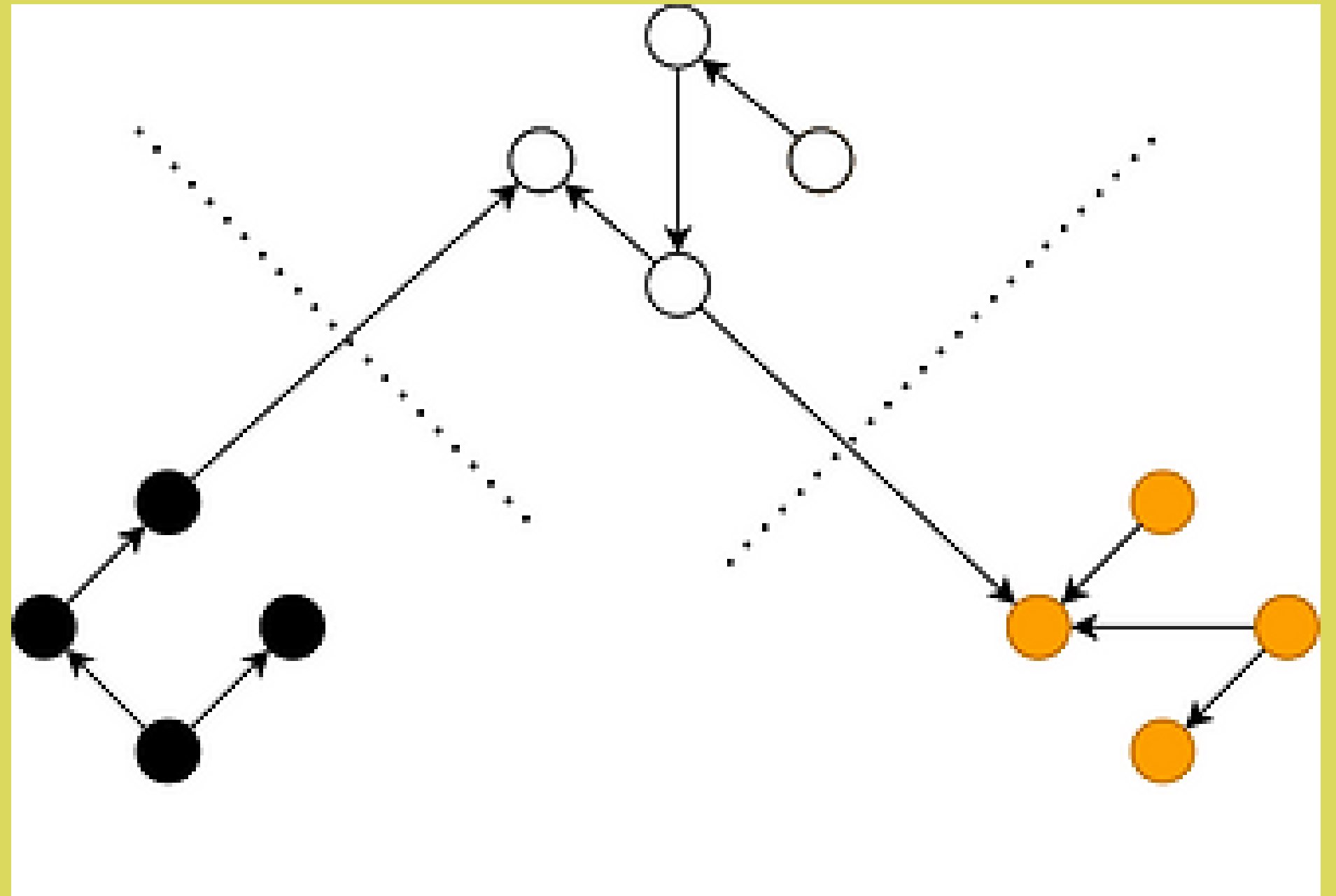
Systems Thinking

A **holistic approach** to software engineering that focuses on the **system as a whole** instead of a collection of isolated components

- It's not just FE vs BE, **it's both** of these combined

The Ideal: Loose coupling, High cohesion

- Clean code
- Easy to maintain
- Can add features/make changes much easier



Architecture vs Design Patterns

Architecture is the scaffolding that everything else is built on top of

Design patterns focus on how to structure the internal workings to solve problems

Common patterns in FSSE

- **Singleton**
- **Observer**
- **Builder**
- **Strategy**

Application Architecture

Patterns: MV*

Common Terms

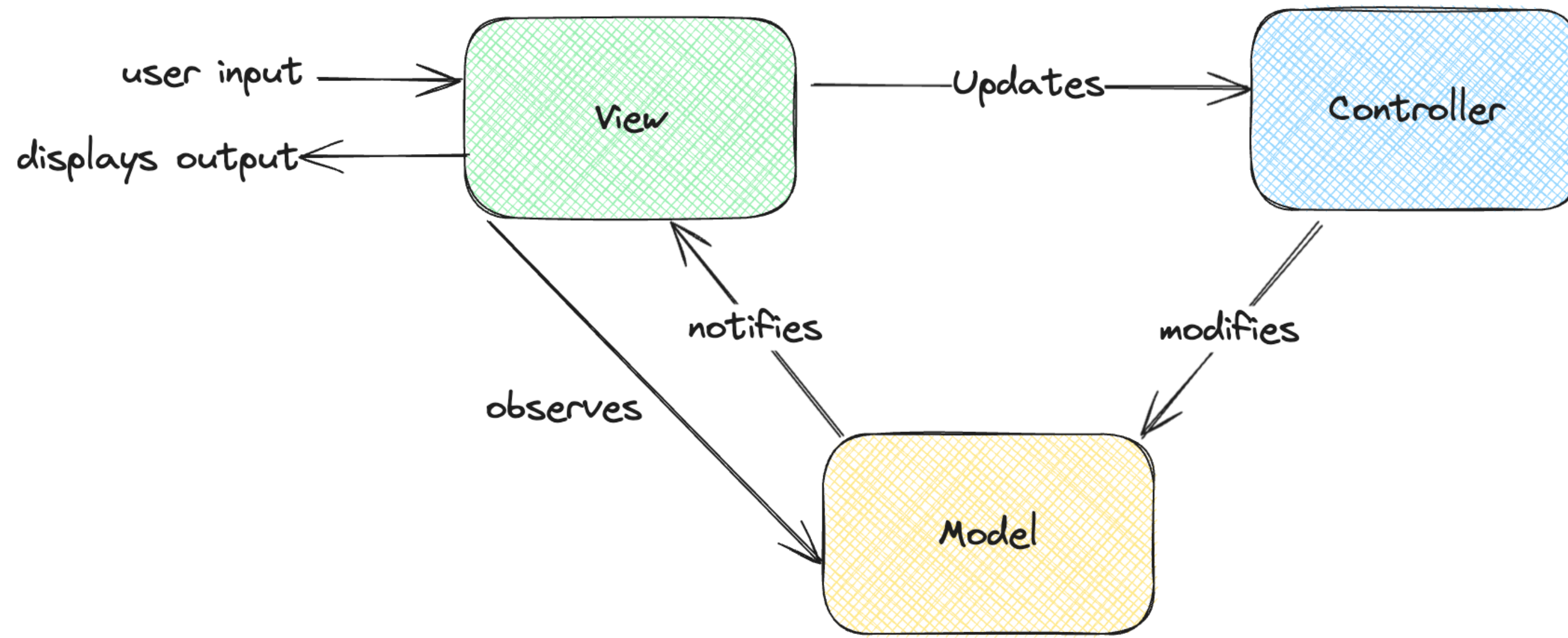
Model - manages data for an app

View - visual representation of the model

Application Architecture Patterns: MV*

Model View Controller (MVC)

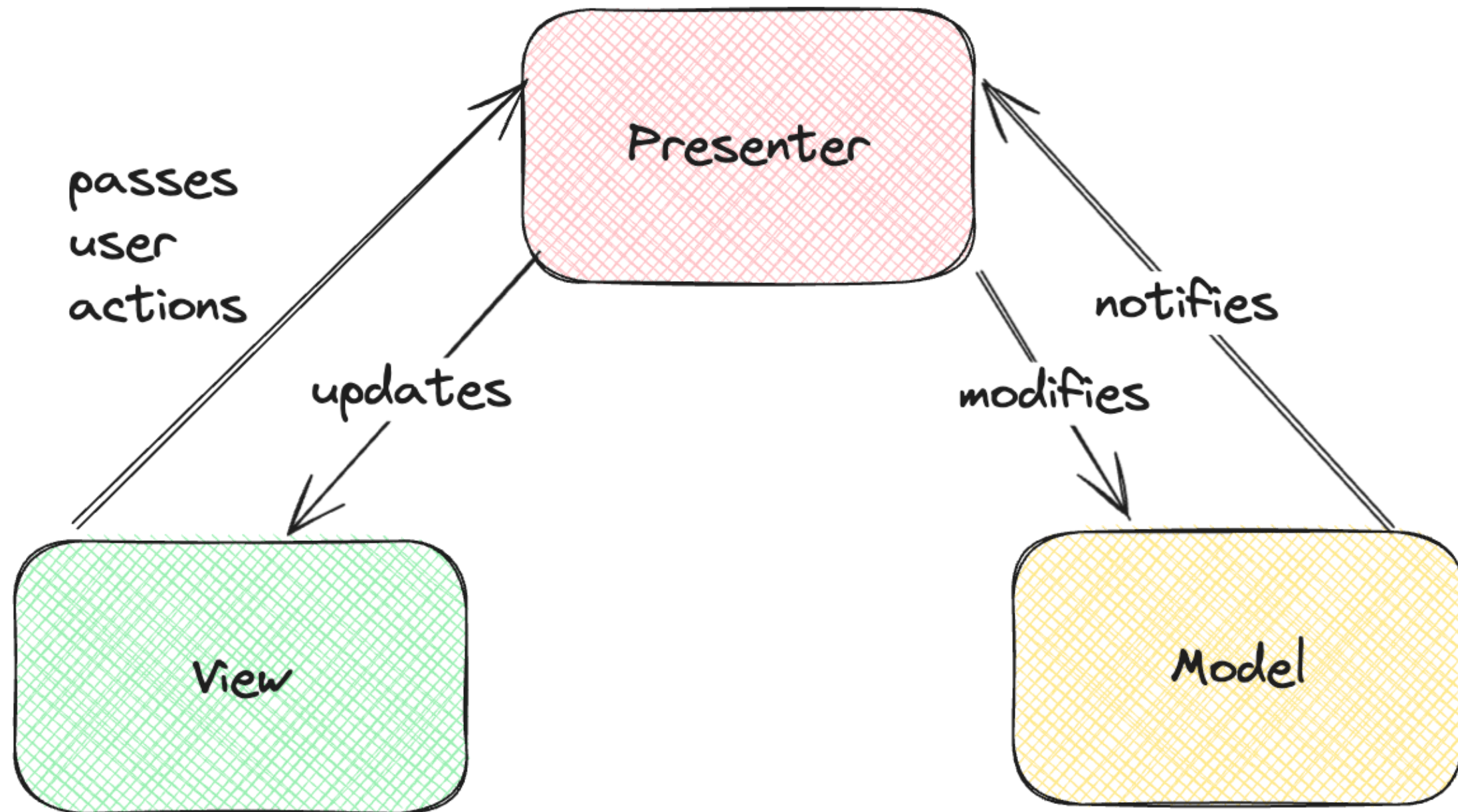
Controllers - in charge of updating the model



Application Architecture Patterns: MV*

Model View Presenter

Presenter - handles modifying the model and the view

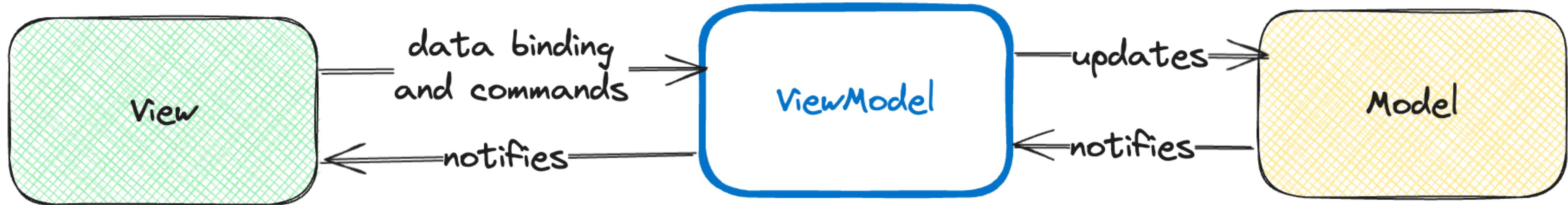


Application Architecture Patterns: MV*

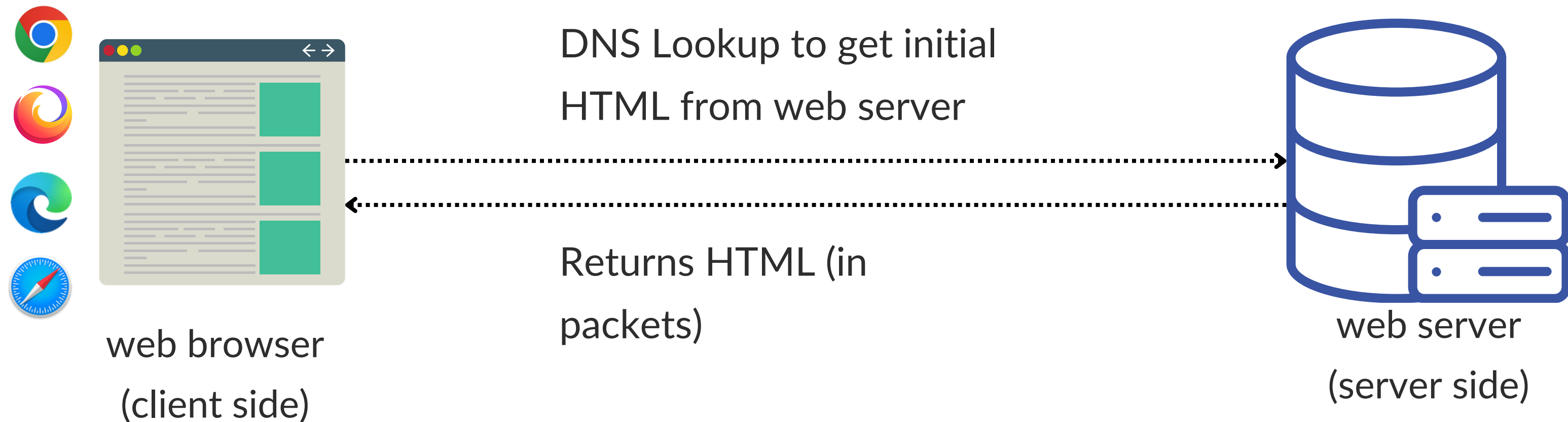
Model View ViewModel

ViewModel data converter with methods exposed to view

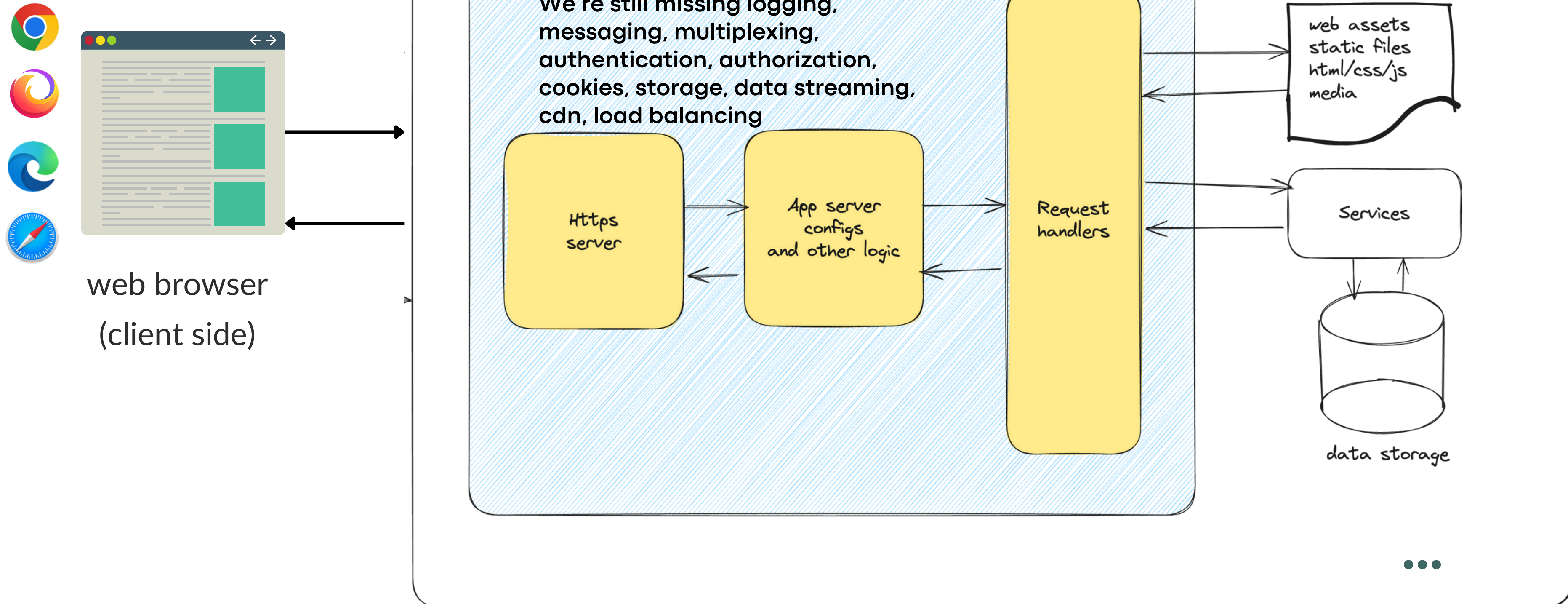
View is active



The web app that we all know...



The web app that we all know...



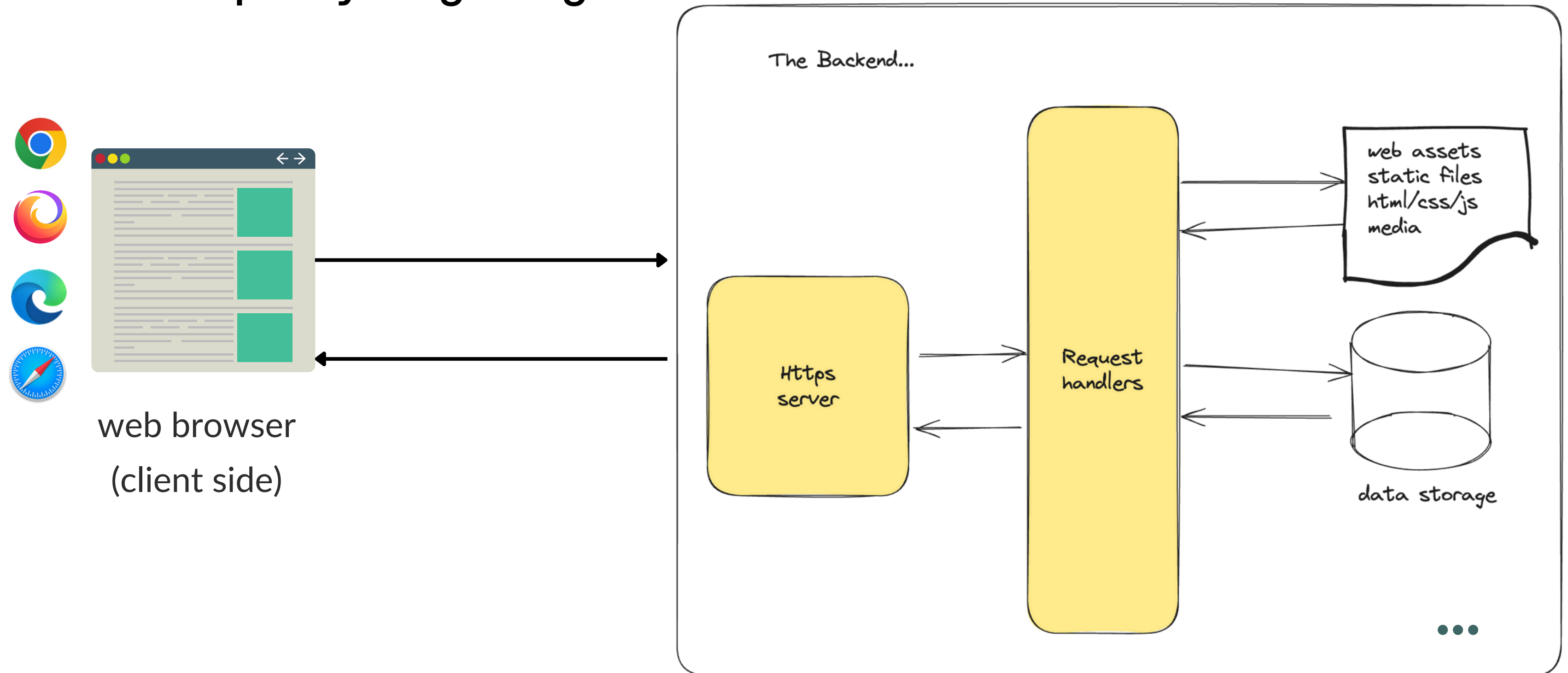
Architecture Patterns for the BE

- SPA (Thin server architecture)
- Microservices
- Serverless
- Layered
- Event based/driven* (we'll discuss next week with sockets)

Patterns for the BE

SPA (Thin server architecture)

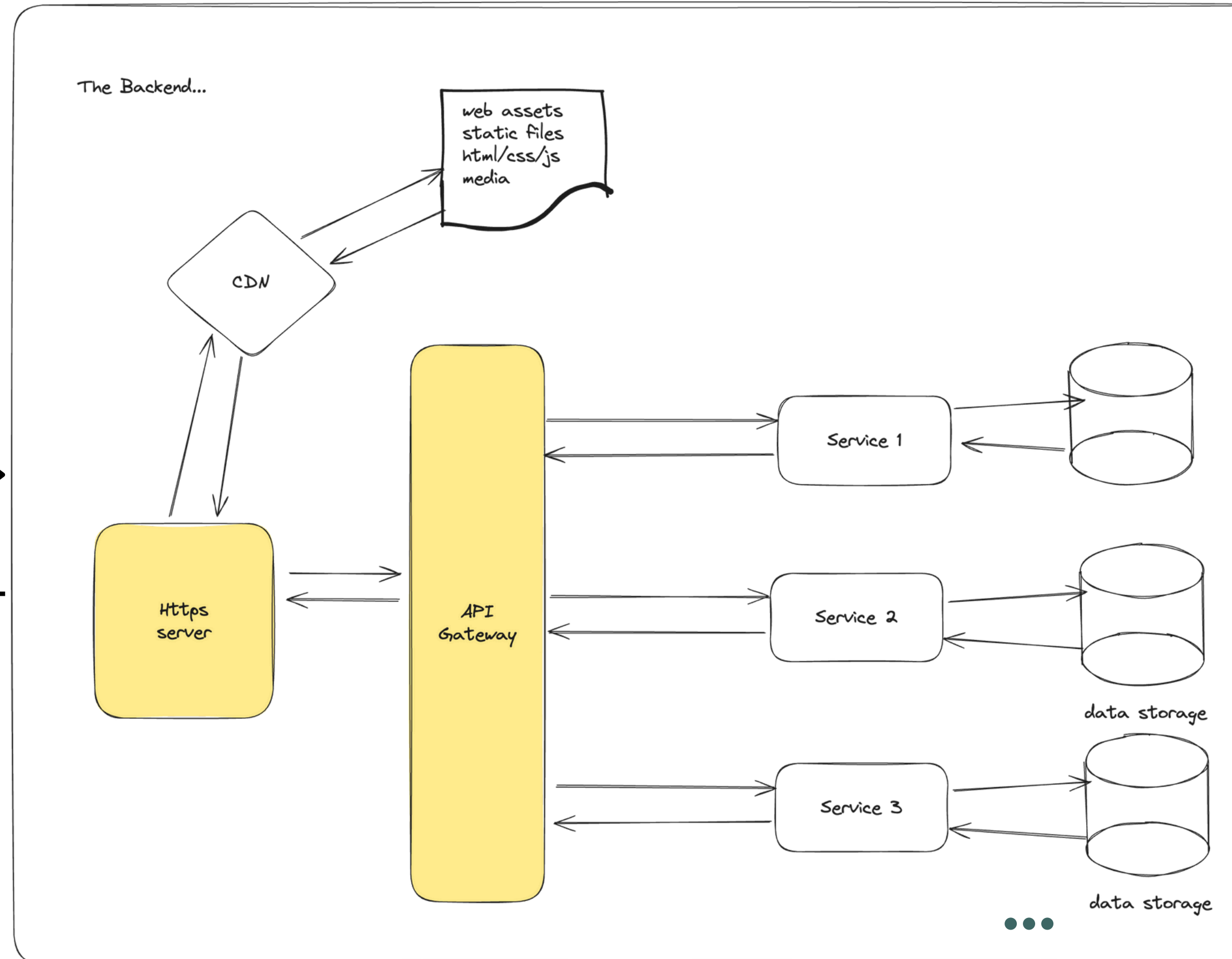
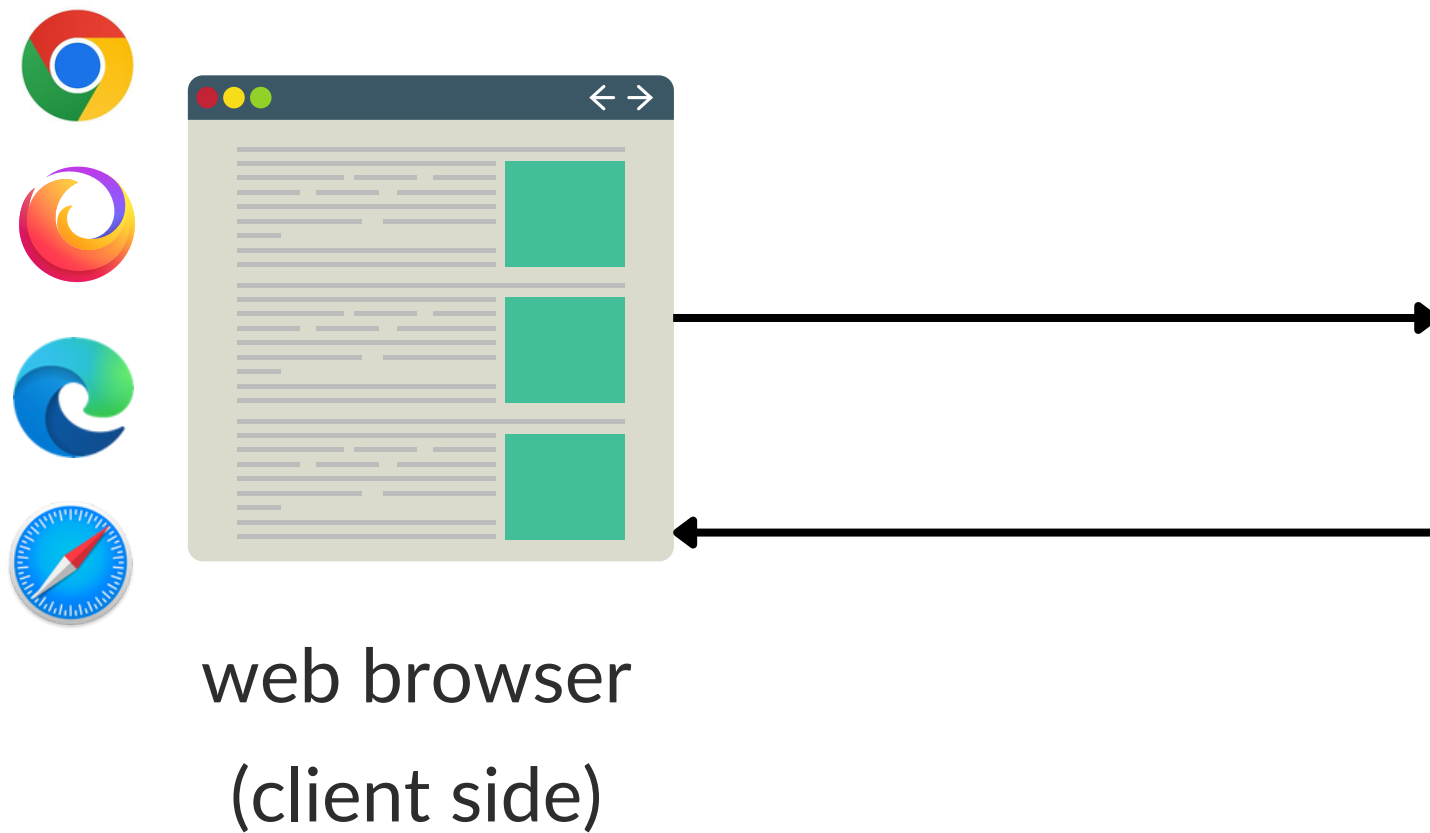
- client performs a lot of the business logic
- server is purely for getting data



Patterns for the BE

Microservices

- When you break up a large app into smaller independent pieces
- Each service has one purpose



Architecture Patterns for the BE

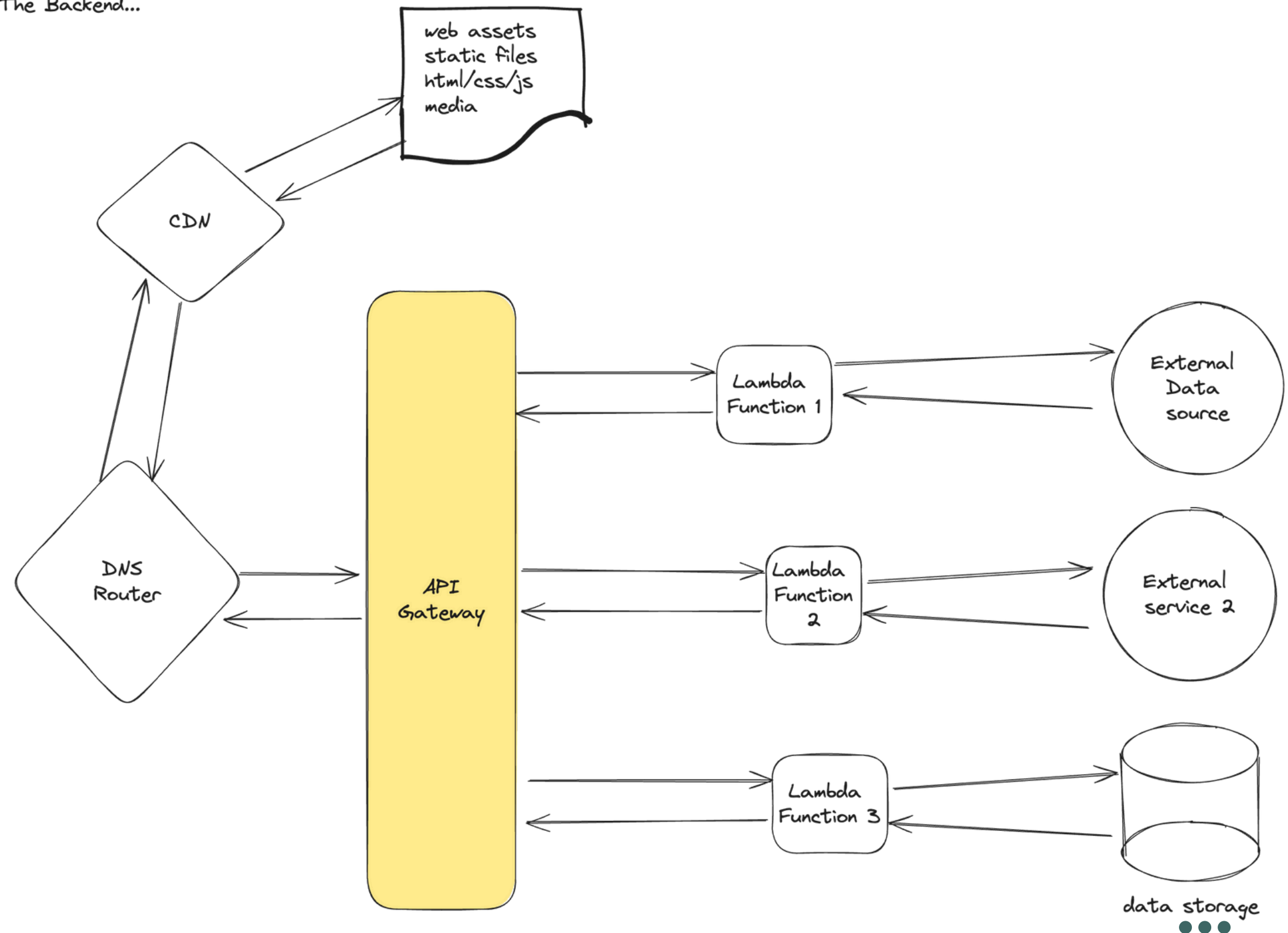
Serverless

- Infra managed by cloud service provider (ex: AWS) and lambdas are used to manage resources



web browser
(client side)

The Backend...



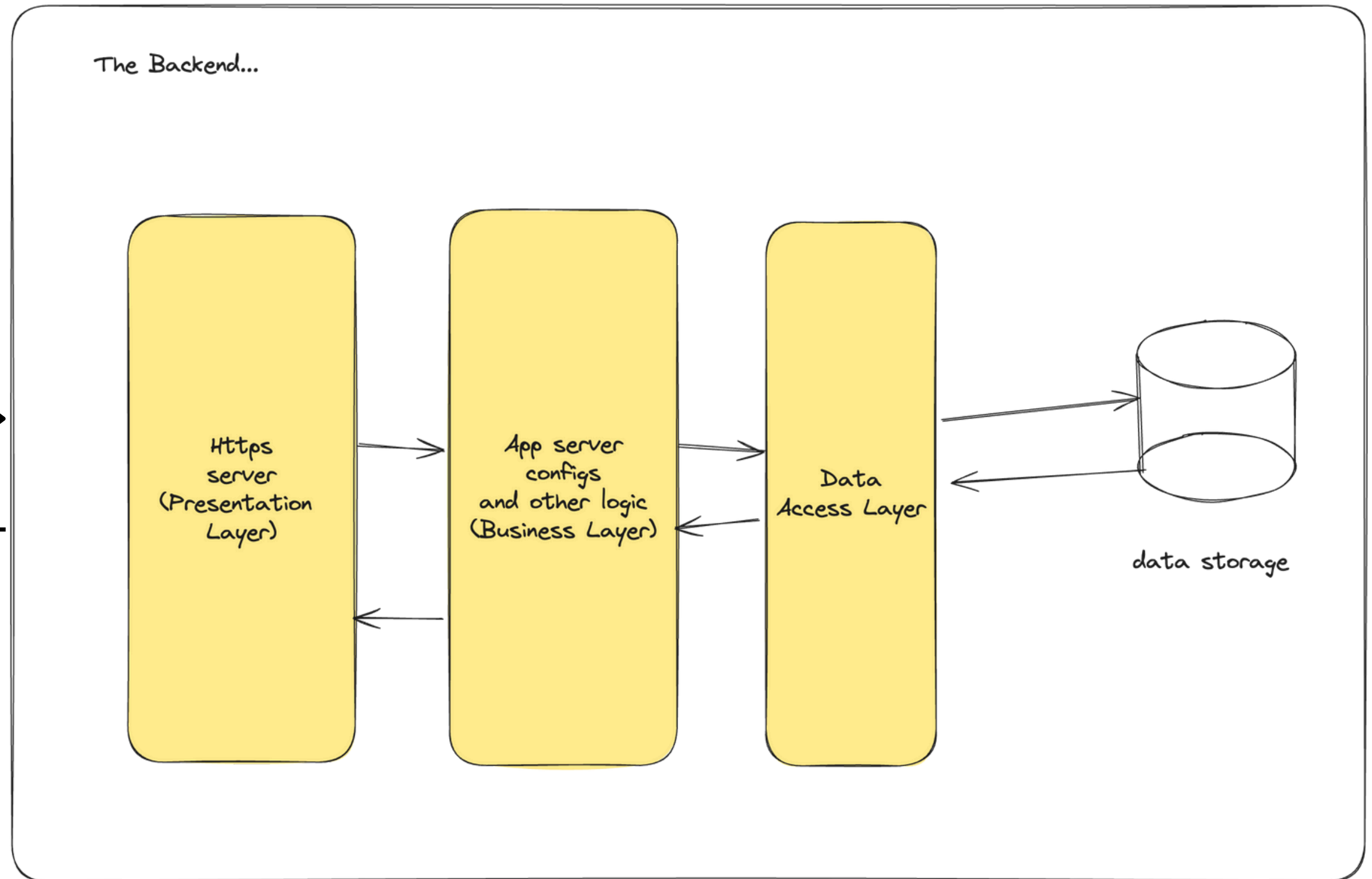
Architecture Patterns for the BE

Layered (monolithic)

- Entire app is one unit with multiple tiers



web browser
(client side)



Frameworks for the BE

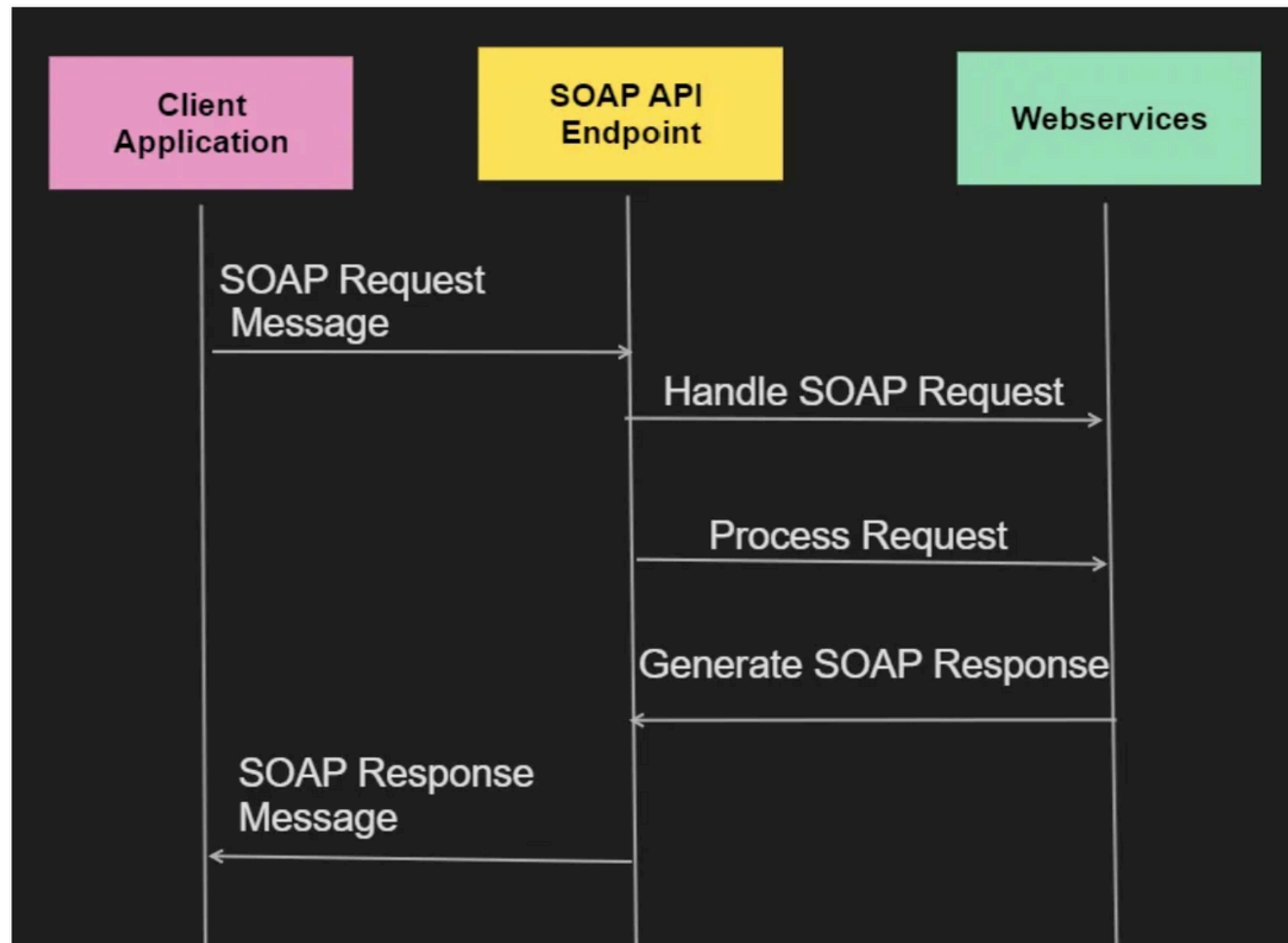
- Django (Python)
- Express (JS)
- Spring boot (Java)
- Ruby on Rails (Ruby)

API FORMATS

- SOAP
- gRPC
- GraphQL
- REST(ful)
- WebSocket
- Webhooks

SOAP

- Simple Object Access Protocol



SOAP

```
<?xml version="1.0"?>
<soap:Envelope
  xmlns:soap="http://www.w3.org/2003/05/soap-envelope/"
  soap:encodingStyle="http://www.w3.org/2003/05/soap-encoding">
  <soap:Body xmlns:m="https://www.superglobamegacorp.example/stock">
    <m:GetStockPriceResponse>
      <m:StockSymbol>AAPL</m:Price>
      <m:Price>170.41</m:Price>
    </m:GetStockPriceResponse>
  </soap:Body>
</soap:Envelope>
```

REST

```
{
  "stockSymbol": "AAPL",
  "price": 170.41
}
```

GraphQL

- Represents data in a graph that's defined by a schema lang
- Is a specification which means it's language neutral

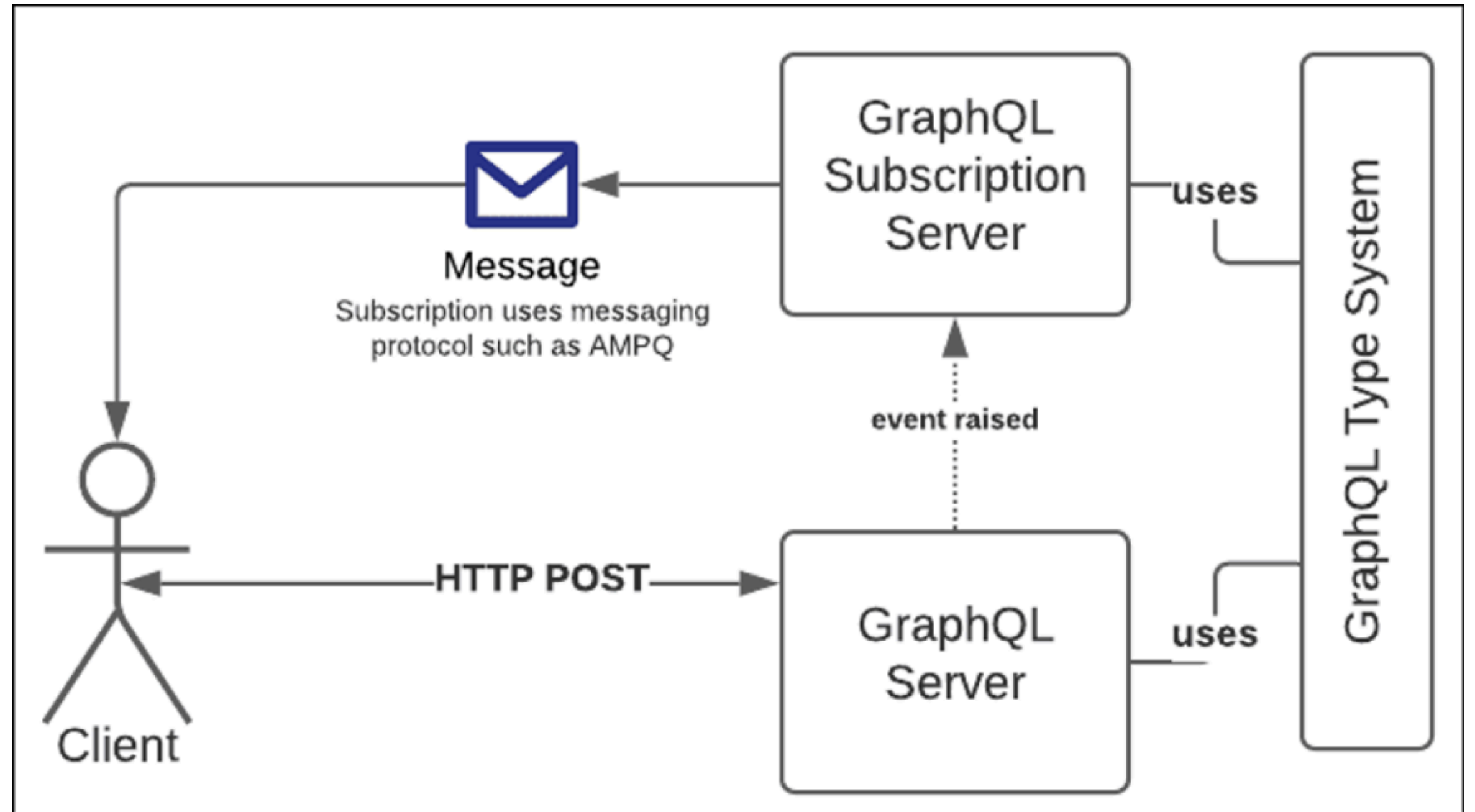
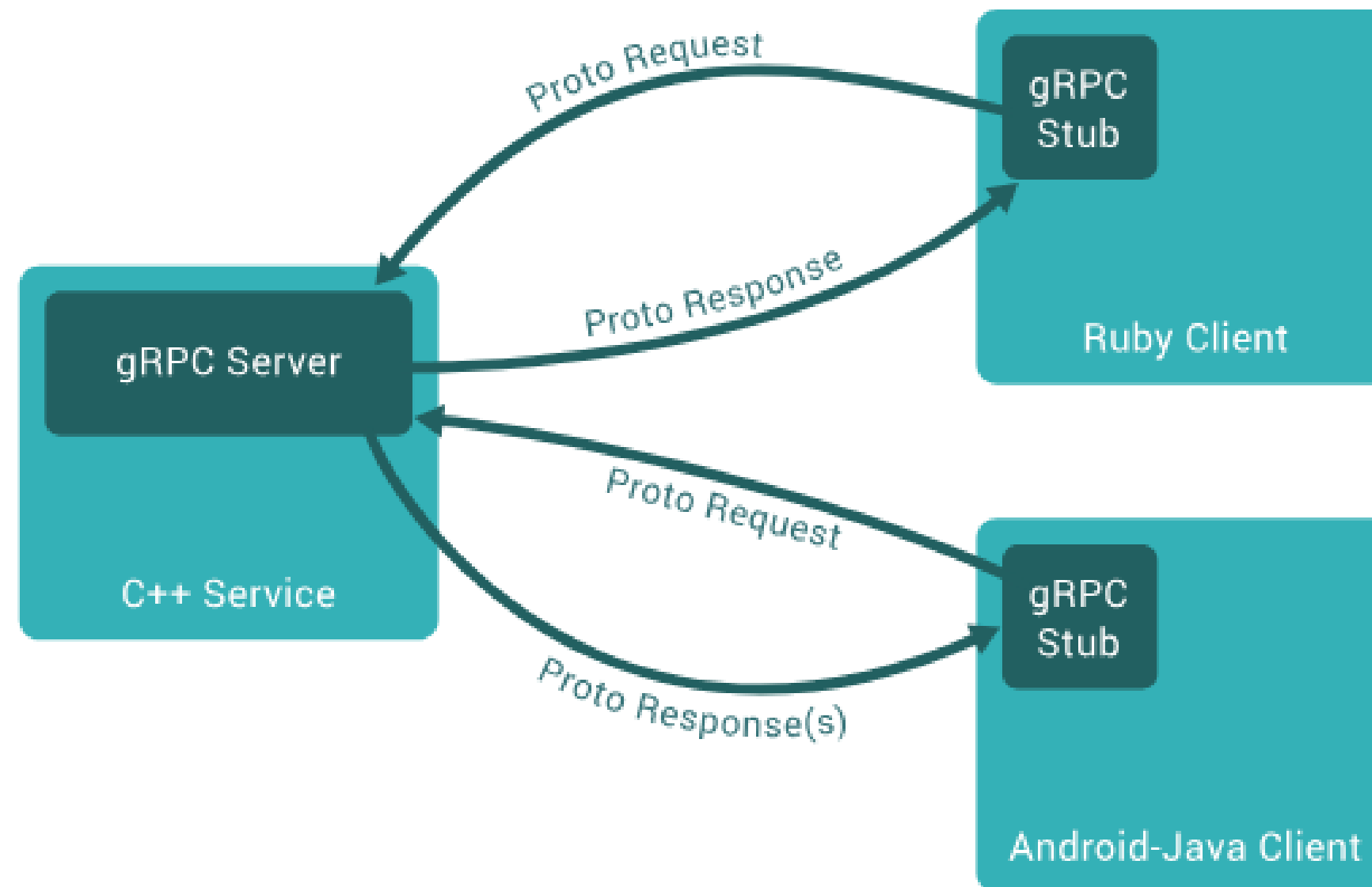


Image source: <https://www.redhat.com/architect/apis-soap-rest-graphql-grpc>

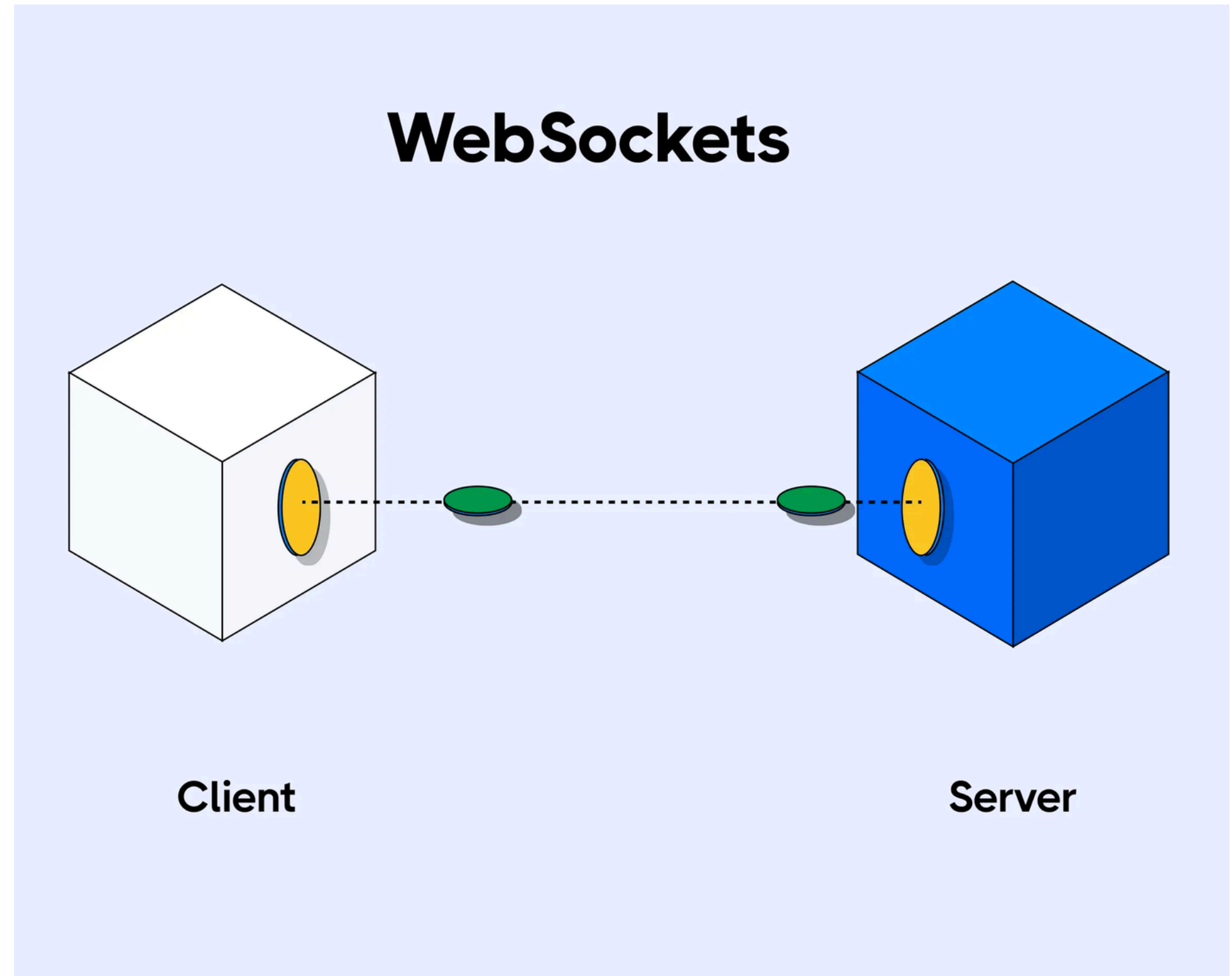
gRPC

- RPC = remote procedure call
- Uses protocol buffers (protos)
- Used by Netflix, developed by Google



WebSockets

- Used to facilitate two-way communication. A socket is open on both client and server
- Scenario: you want two-way communication between networked systems, like a chat application



WebHooks

- Used when sending requests to another application. A source and a destination applications
- The request might contain data on the event that was triggered. That request data is used in the event processing
- Used only with applications
- One-way communication with socket closing once complete

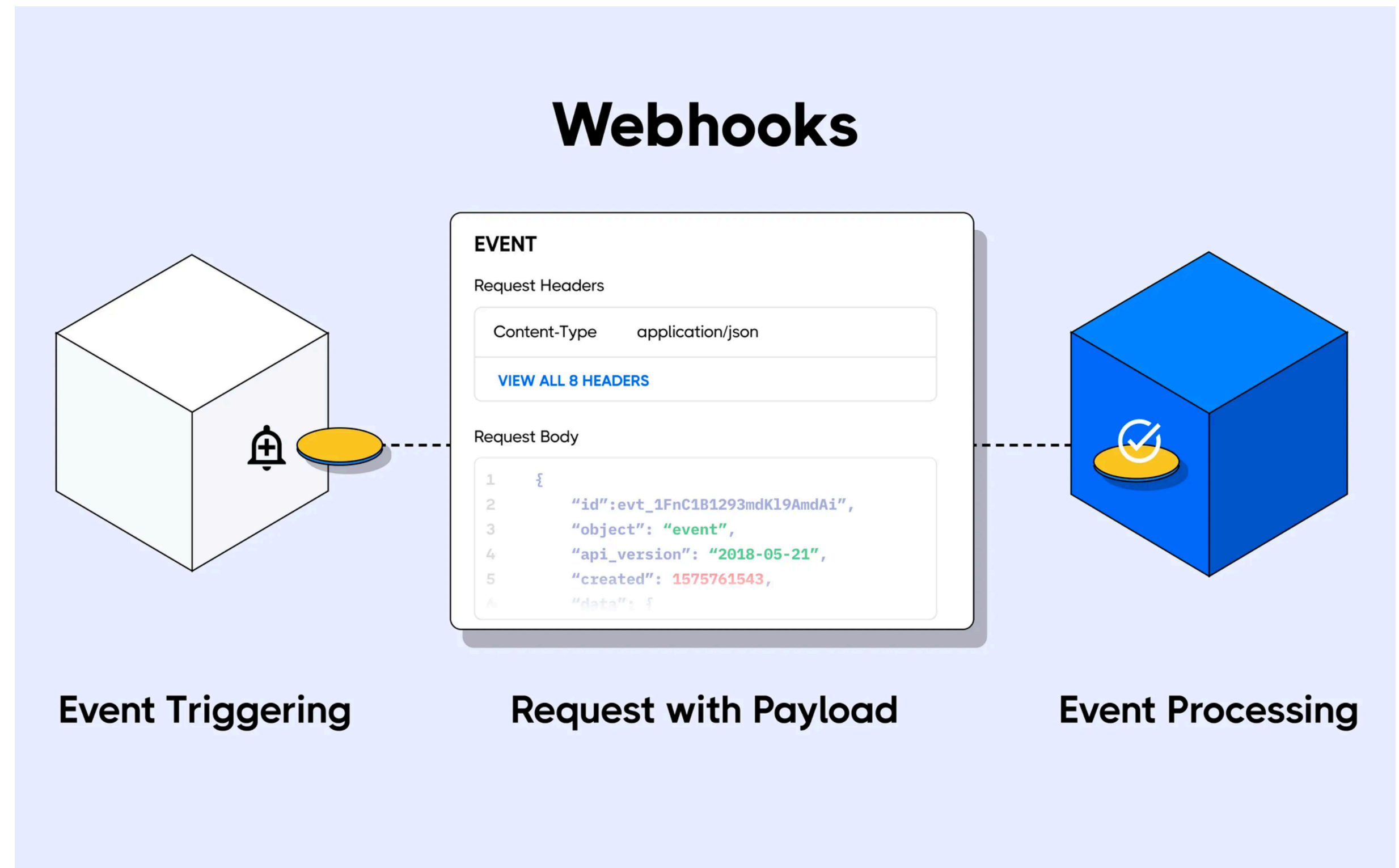


Image source: <https://hookdeck.com/webhooks/guides/when-to-use-webhooks>

API FORMATS

- SOAP
- gRPC
- GraphQL
- REST(ful)
- WebSocket
- Webhooks

What is REST?



Representational State Transfer - architectural style for APIs (not a protocol)

Representational - resource is a representation of what the server side has

State - client requests move server through different states

Transfer - result of state transition gets returned to the client

REST(ful)

- Representational State Transfer

Constraints:

1. Uniform Interface
2. Stateless Operations
3. Layered System
4. Client-server based
5. RESTful resource caching
6. Code on demand

**RESTful APIs adhere to all
REST constraints**



REST(ful)

2. Stateless Operations

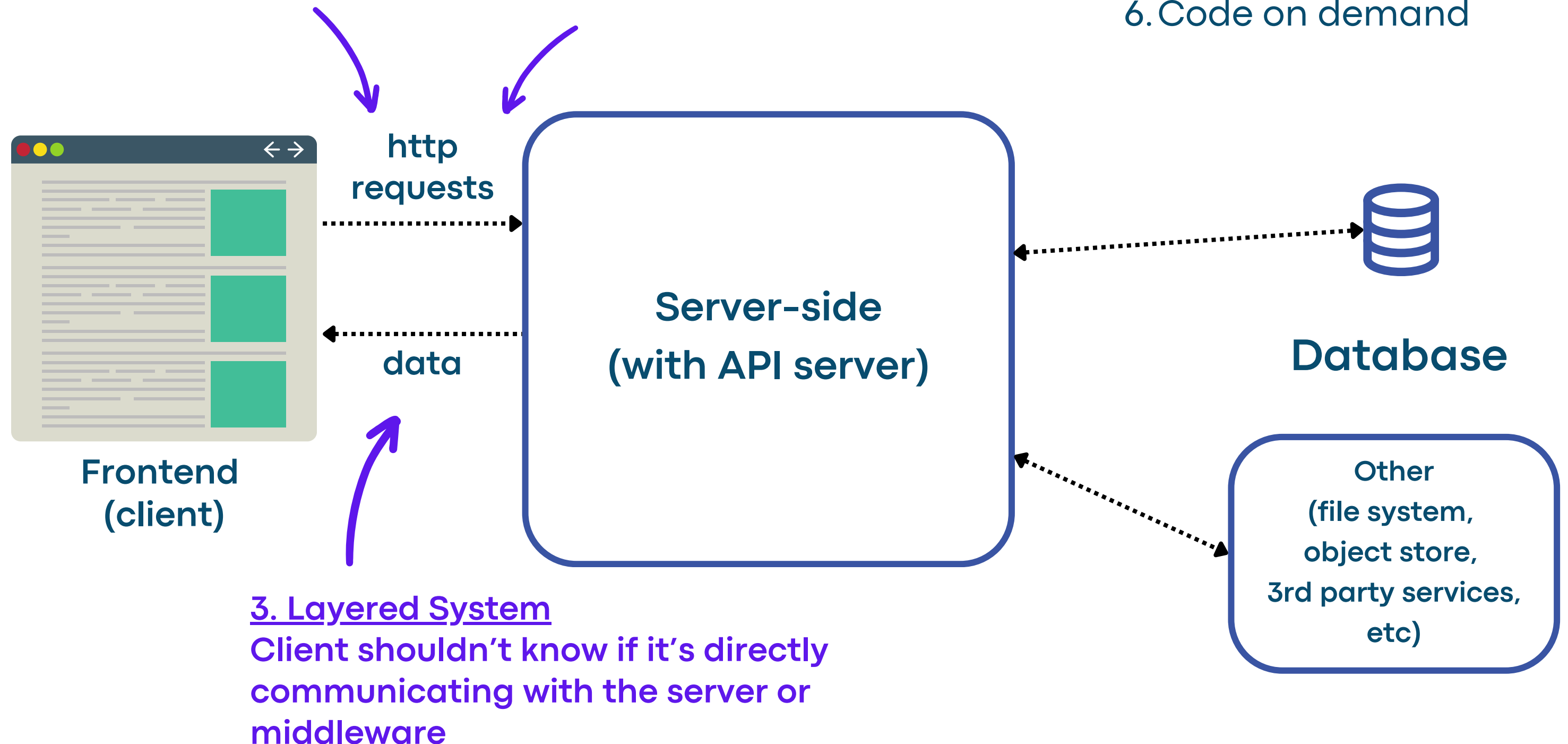
- All info that my server needs to know should be in the request
- Doesn't need previous request info

1. Uniform Interface

Endpoints refer to only one resource

Constraints:

1. Uniform Interface
2. Stateless Operations
3. Layered System
4. Client-server based
5. RESTful resource caching
6. Code on demand



REST(ful)

1. Uniform Interface

Endpoints refer to only one resource

`/users`

`/users/:userId`

`/posts`

`/posts/:postId`

2. Stateless Operations

- All info that my server needs to know should be in the request
- Doesn't need previous request info
 - hence pagination!

`/posts?offset=0&limit=20`

`/posts?offset=20&limit=20`

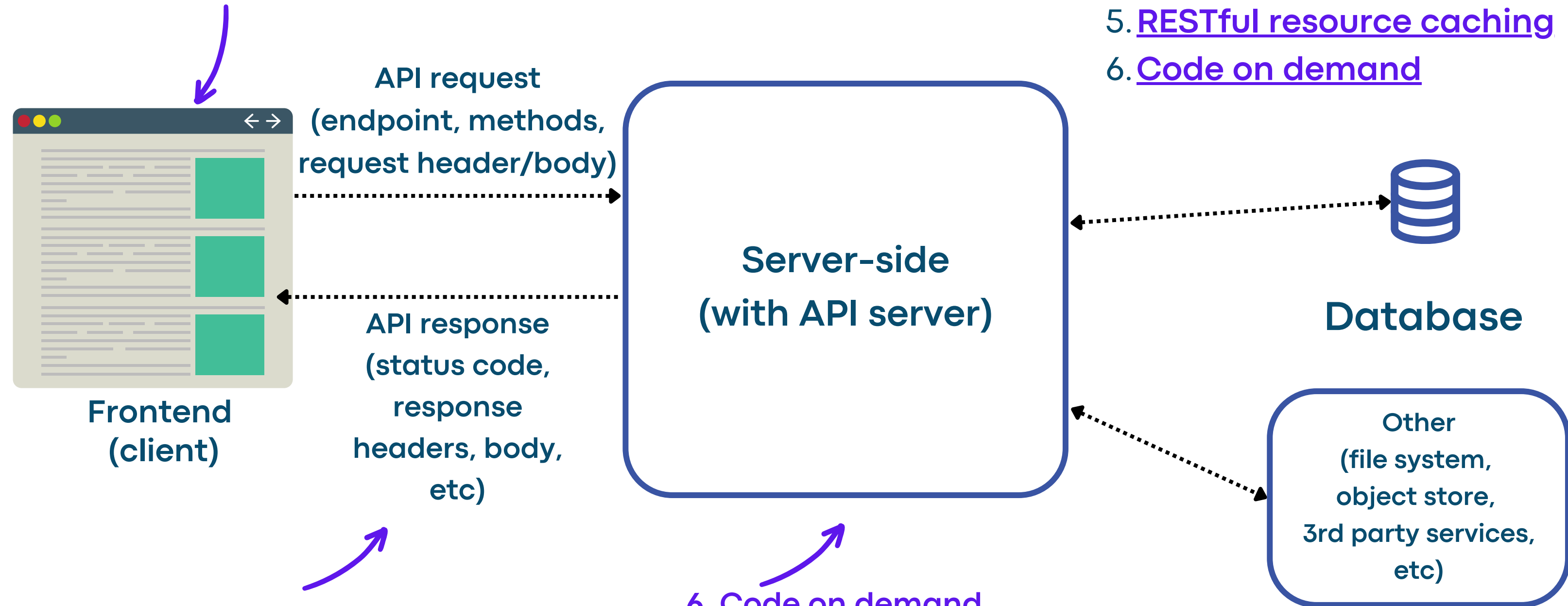
REST(ful)

4 Client-server based

UI is separate from data
request/storage

Constraints:

1. Uniform Interface
2. Stateless Operations
3. Layered System
4. Client-server based
5. RESTful resource caching
6. Code on demand



5. RESTful resource caching

Response data needs to be labeled
as cacheable or not

6. Code on demand

Server can send client
executable code as
needed



Why REST?

- REST provides a standard so that any server can talk to any other server
- Companies started creating APIs so that markets could be expanded
- Provide a lot of flexibility
- Scalable (separation between server and client)
- Support for multiple formats (JSON, XML, and HTML)



Why NOT REST?

- Slow...
- Not great for aggregating multiple resources (data sources)
- Doesn't evolve well over time

Request Methods



Aka HTTP verbs

POST - to create/submit data entity

- Create

GET - requests a specific resource (only meant to retrieve data)

- Read

PUT - replace/update the existing entity

- Update

DELETE - deletes existing entity

- Delete



Endpoints

Tips on naming endpoints:

1. Use nouns
2. Plural vs singular?
3. Don't state your CRUD verb in the endpoint
a.ex: `http://api.example.com/update_customer/12345`

Anatomy of a Request

Includes:

Request method

Doc path

Http version

Additional info for what type of data is being requested (ex: language, caching, etc)

Can have data block for POST requests

▼ GET / HTTP/1.1

Host google.com

User-Agent HTTPie

▼ POST /api/recipe-steps HTTP/1.1

Content-Length 52

Content-Type application/json

Host localhost:8080

User-Agent HTTPie

```
▼ {  
  "step": "Another step 2",  
  "completed": true  
}
```

Anatomy of a Response



```
▼ HTTP/1.1 200 OK

Accept-Ranges      none
Cache-Control      private, max-age=0
Connection         close
Content-Security-Policy-Report-Only
  object-src 'none';base-uri 'self';script-src 'nonce-2jejjGsnQ79DqV
  g_1tEDsg' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-i
  nline' https: http::report-uri https://csp.withgoogle.com/csp/gws/
  other-hp
Content-Type       text/html; charset=ISO-8859-1
Date               Mon, 05 Feb 2024 04:39:01 GMT
Expires            -1
P3p                CP="This is not a P3P policy! See g.co/p3phelp for more info."
Server             gws
Set-Cookie         1P_JAR=2024-02-05-04; expires=Wed, 06-Mar-2024 04:39:01 GMT; path
  =/; domain=.google.com; Secure, AEC=Ae3NU90wePlpJvyz38pnpz1Sqknrm
  Hf4He2-_o_p0MR1aBCIXXRkST-eM0; expires=Sat, 03-Aug-2024 04:39:01 G
  MT; path=/; domain=.google.com; Secure; HttpOnly; SameSite=lax, NI
  D=511=kveNOT5fTmPF25XX3B3Y70t7hgHP0GiEVwkxgl09eiAa_0X4H9v9_oUgHGIp
  wcdJL5NuqKJMvoeP_7ngLm4iosTssjlXksdQMRjiFGmSc4AUHPdkNAIdw5-98RydmW
  4iVoSzFRAj7s5TcuBTUUhEbEUXo3BHBeB_p1hlT4ihv0s; expires=Tue, 06-Aug
  -2024 04:39:01 GMT; path=/; domain=.google.com; HttpOnly

Transfer-Encoding  chunked
Vary               Accept-Encoding
X-Frame-Options    SAMEORIGIN
X-Xss-Protection   0

▼ <!doctype html><html itemscope="" itemtype="http://schema.org/WebPage" lang="en"><head><meta
  content="Search the world's information, including webpages, images, videos and more. Google has
  many special features to help you find exactly what you're looking for." name="description"><meta
  content="noodp" name="robots"><meta content="text/html; charset=UTF-8" http-equiv="Content-Type">
  <meta content="/images/branding/googleg/1x/googleg_standard_color_128dp.png" itemprop="image">
```

```
▼ HTTP/1.1 201 Created

Connection         close
Content-Length      0
Date               Mon, 05 Feb 2024 04:45:08 GMT
```

Includes:

- Http version with status code
- Info about the data that's being sent in the response
- Data (ex: the html)

HTTP Status Codes



5 categories:

1xx -- informational

2xx -- successful

3xx -- redirection

4xx -- client error

5xx -- server error

Key Components of a BE Framework

- Routing
- Authentication/Authorization
- Template engine (though this can usually be handled by FE framework)
- Db integration

Django Code Along

Web framework built in python



Used by:

- Instagram
- Pinterest
- Disqus
- Mozilla
- YouTube
- Spotify

Spring Boot



Framework dedicated to building web applications (and REST apis) that sits on top of Spring

Very opinionated

Common Terms:

- Beans
- Annotations
- Data Repositories
- Inversion of Control